Assignment: 3

NAME: Nikita Sandip Patil
PRN No: 125B2F005
Subject: Database Management System
Div: IT A4

1. Aim:

SQL Queries- All types of Join, Sub-Query and View:

a. Write at least 10 SQL queries for suitable database application using SQL DML statements using joins and subqueries.
b. Write queries using group by and having clause.

2. Theory

- **Joins** combine data from multiple tables.
- **Subqueries** are queries inside another query.
- **GROUP BY** groups records.
- **HAVING** filters grouped data.

3. Database and Table Creation

```
CREATE DATABASE CompanyDB;
USE CompanyDB;

CREATE TABLE Department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50),
    location VARCHAR(50)
);

CREATE TABLE Employee (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50),
    salary INT,
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
```

Data Insertion

```
INSERT INTO Department VALUES
(1, 'HR', 'Pune'),
(2, 'IT', 'Mumbai'),
(3, 'Finance', 'Delhi'),
(4, 'Marketing', 'Pune');

INSERT INTO Employee VALUES
(101, 'Nikita', 32000, 1),
(102, 'Vedant', 25000, 2),
(103, 'Amit', 35000, 2),
(104, 'Sneha', 28000, 3),
(105, 'Ravi', 40000, 1),
(106, 'Neha', 22000, NULL);
```

Part (a): Joins and Subqueries

1. INNER JOIN

```
SELECT e.emp_name, e.salary, d.dept_name
FROM Employee e
INNER JOIN Department d
ON e.dept_id = d.dept_id;
```

```
+----------------+-------------+------------------+
| emp_name | salary | dept_name |
+----------------+-------------+------------------+
| Nikita   | 32000  | HR        |
| Ravi     | 40000  | HR        |
| Vedant   | 25000  | IT        |
| Amit     | 35000  | IT        |
| Sneha    | 28000  | Finance   |
+----------------+-------------+------------------+
```

2. LEFT JOIN

```
SELECT e.emp_name, d.dept_name
FROM Employee e
LEFT JOIN Department d
ON e.dept_id = d.dept_id;
```

```
+----------------+------------------+
| emp_name | dept_name |
+----------------+------------------+
| Nikita    | HR         |
| Vedant    | IT         |
| Amit      | IT         |
| Sneha     | Finance    |
| Ravi      | HR         |
| Neha      | NULL       |
+----------------+------------------+
6 rows in set (0.00 sec)
```

3. RIGHT JOIN

```
SELECT e.emp_name, d.dept_name
FROM Employee e
RIGHT JOIN Department d
ON e.dept_id = d.dept_id;
```

```
+----------------+------------------+
| emp_name | dept_name |
+----------------+------------------+
| Nikita    | HR         |
| Ravi      | HR         |
| Vedant    | IT         |
| Amit      | IT         |
| Sneha     | Finance    |
| NULL      | Marketing |
+----------------+------------------+
```

4. SELF JOIN

```
SELECT e1.emp_name, e2.emp_name, e1.dept_id
FROM Employee e1
JOIN Employee e2
ON e1.dept_id = e2.dept_id
WHERE e1.emp_id <> e2.emp_id;
```

```
+----------------+----------------+---------------+
| emp_name | emp_name | dept_id |
+----------------+----------------+---------------+
| Nikita    | Ravi      |      1    |
| Vedant    | Amit      |      2    |
| Amit      | Vedant    |      2    |
```

```
| Ravi      | Nikita    |      1    |
+----------------+----------------+---------------+
4 rows in set (0.00 sec)
```

5. Subquery (Above Average Salary)

SELECT emp_name, salary
FROM Employee
WHERE salary > (SELECT AVG(salary) FROM Employee);

```
+----------------+-------------+
| emp_name | salary |
+----------------+-------------+
| Nikita    | 32000  |
| Amit      | 35000  |
| Ravi      | 40000  |
+----------------+-------------+
```

6. Subquery (HR Department)

SELECT emp_name
FROM Employee
WHERE dept_id = (
  SELECT dept_id FROM Department WHERE dept_name = 'HR');

```
+----------------+
| emp_name |
+----------------+
|  Nikita  |
|  Ravi    |
+----------------+
```

7. Correlated Subquery

```
SELECT emp_name, salary, dept_id
FROM Employee e1
WHERE salary = (
    SELECT MAX(salary)
    FROM Employee e2
    WHERE e1.dept_id = e2.dept_id);
```

```
+----------------+-------------+---------------+
| emp_name | salary | dept_id |
+----------------+-------------+---------------+
```

```
| Amit      | 35000  |    2    |
| Sneha     | 28000  |    3    |
| Ravi      | 40000  |    1    |
+----------------+-------------+---------------+
```

8. EXISTS Subquery

```
SELECT emp_name
FROM Employee e
WHERE EXISTS (
   SELECT *
   FROM Department d
   WHERE e.dept_id = d.dept_id
   AND d.location = 'Pune');
```

```
+----------------+
| emp_name |
+----------------+
|   Nikita  |
|   Ravi   |
+----------------+
```

9. JOIN + GROUP BY

```
SELECT d.dept_name, SUM(e.salary) AS total_salary
FROM Employee e
JOIN Department d
ON e.dept_id = d.dept_id
GROUP BY d.dept_name;
```

```
+------------------+----------------------+
| dept_name | total_salary |
+------------------+----------------------+
| HR         | 72000        |
| IT         | 60000        |
| Finance    | 28000        |
+------------------+----------------------+
```

10. Subquery using IN

```
SELECT emp_name
FROM Employee
WHERE dept_id IN (
    SELECT dept_id FROM Department WHERE location = 'Mumbai');
```

```
+ --------- +
| emp_name |
+ --------- +
|  Vedant  |
|   Amit   |
+ --------- +
```

Part (b): GROUP BY and HAVING

1. Count Employees

```
SELECT dept_id, COUNT(*) AS total_employees
FROM Employee
GROUP BY dept_id;
```

```
+---------------+---------------------------+
| dept_id | total_employees |
+---------------+---------------------------+
|   NULL  |       1         |
|     1   |       2         |
|     2   |       2         |
|     3   |       1         |
+---------------+---------------------------+
```

2. HAVING Count > 1

```
SELECT dept_id, COUNT(*) AS total_employees
FROM Employee
GROUP BY dept_id
HAVING COUNT(*) > 1;
```

```
+---------------+---------------------------+
| dept_id | total_employees |
+---------------+---------------------------+
|    1    |     2           |
|    2    |     2           |
+---------------+---------------------------+
```

3. Average Salary

```
SELECT dept_id, AVG(salary) AS avg_salary
FROM Employee
GROUP BY dept_id;
```

```
+---------------+------------------+
| dept_id | avg_salary|
```

```
+---------------+------------------+
|    NULL | 22000.0000 |
|       1 | 36000.0000 |
|       2 | 30000.0000 |
|       3 | 28000.0000 |
+---------------+------------------+
```

4. HAVING AVG > 30000

```
SELECT dept_id, AVG(salary) AS avg_salary
FROM Employee
GROUP BY dept_id
HAVING AVG(salary) > 30000;
+---------------+------------------+

| dept_id | avg_salary|
+---------------+------------------+
|   1     | 36000     |
+---------------+------------------+
```

5. HAVING SUM > 50000

```
SELECT dept_id, SUM(salary) AS total_salary
FROM Employee
GROUP BY dept_id
HAVING SUM(salary) > 50000;
+---------------+------------------+

| dept_id | avg_salary|
+---------------+------------------+
|    1    |   72000   |
|    2    |   60000   |
+---------------+------------------+
```

---

4. Conclusion:

This assignment successfully demonstrates SQL operations using joins, subqueries, GROUP BY, and HAVING clauses along with their outputs.